



LeetCode 901: Online Stock Span

1. Problem Title & Link

Title: LeetCode 901: Online Stock Span

Link: <https://leetcode.com/problems/online-stock-span/>

2. Problem Statement (Short Summary)

Design a class:

StockSpanner

that supports:

next(price)

For each day's stock price, return the stock span.

Stock Span =

Number of consecutive days

(including today)

for which price $\leq$ today's price

3. Examples (Input → Output)

Example 1

Input:

["StockSpanner","next","next","next","next","next","next","next"]

[[],[100],[80],[60],[70],[60],[75],[85]]

Output:

[null,1,1,1,2,1,4,6]

Explanation

Price	Span
100	1
80	1
60	1
70	2
60	1
75	4
85	6

Example 2

Input:



100, 101, 102

Output:

1, 2, 3

Example 3

Input:

50, 40, 30

Output:

1, 1, 1

4. Constraints

- $1 \leq \text{price} \leq 10^5$
- At most 10^4 calls to next()

Need efficient processing for every query.

5. Core Concept (Pattern / Topic)

Monotonic Stack

Related Topics:

- Next Greater Element
- Stack
- Span Problems

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every new price:

Move backward.

Keep counting

until a larger price appears

Example:

100 80 60 70

For:

70

Need to check:

60 ✓

80 ✗

Span:



2

Complexity:

$O(n)$ per query

Total:

$O(n^2)$

Too slow.

Key Observation

When a larger price arrives:

75

Previous smaller prices:

60

70

will never be useful again.

We can remove them.

This suggests:

Monotonic Decreasing Stack

What To Store?

Store:

(price, span)

instead of only price.

Example:

[(100,1)]

New price:

80

Stack:

[(100,1),(80,1)]

New price:

70

Stack:

[(100,1),(80,1),(70,1)]

New price:

75

Pop:

70

and accumulate span.



Why It Works

Each element is:

Pushed once

Popped once

Therefore:

Amortized $O(1)$

per query.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Prices:
100 80 60 70 60 75 85
Stack Evolution:
100

100
80

100
80
60

100
80
70
When:
75
arrives:
Pop 60

Pop 70
Combine spans.
Result:
Span = 4
```

8. Pseudocode (Language Independent)

```
Initialize stack

next(price):

    span = 1

    while stack not empty
        and stack.top.price <= price:
```



```
        span += stack.top().span

        pop stack

    push(price, span)

    return span
```

9. Code Implementation

✓ Python

```
class StockSpanner:

    def __init__(self):

        self.stack = []

    def next(self, price):

        span = 1

        while (
            self.stack and
            self.stack[-1][0] <= price
        ):

            span += self.stack.pop()[1]

        self.stack.append(
            (price, span)
        )

        return span
```

✓ Java

```
import java.util.*;

class StockSpanner {

    private Stack<int[]> stack;

    public StockSpanner() {

        stack = new Stack<>();

    }

    public int next(int price) {
```



```
int span = 1;

while (
    !stack.isEmpty() &&
    stack.peek()[0] <= price
) {

    span += stack.pop()[1];
}

stack.push(
    new int[]{price, span}
);

return span;
}
```

10. Time & Space Complexity

Time Complexity

Each element:

Push once

Pop once

Total:

$O(n)$

for all operations.

Amortized:

$O(1)$

per query.

Space Complexity

$O(n)$

Stack storage.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

<

instead of:

<=



Problem requires:
less than OR equal

Mistake 2

Storing only prices.
Need spans too.

Mistake 3

Recalculating previous spans repeatedly.
Causes $O(n^2)$.

12. Detailed Dry Run (Step-by-Step Table)

Input:

100
80
60
70
60
75
85

Day 1

100
Stack:
[(100,1)]
Span:
1

Day 2

80
No pop.
Stack:
[(100,1),(80,1)]
Span:
1

Day 3



60

Stack:

$[(100,1),(80,1),(60,1)]$

Span:

1

Day 4

70

Pop:

60

Span:

$1+1=2$

Stack:

$[(100,1),(80,1),(70,2)]$

Return:

2

Day 6

75

Pop:

60(span=1)

70(span=2)

Span:

$1+1+2=4$

Return:

4

Day 7

85

Pop:

75(span=4)

80(span=1)

Span:

$1+4+1=6$

Return:



6

13. Common Use Cases (Real-Life / Interview)

Stock Market Analysis

Find continuous bullish streaks.

Weather Monitoring

Count consecutive days below a threshold.

Sales Analytics

Measure growth streaks.

Performance Monitoring

Track consecutive periods of improvement.

14. Common Traps (Important!)

- Using $<$ instead of $<=$
- Forgetting accumulated spans
- Using brute force backward traversal
- Storing only prices

15. Builds To (Related LeetCode Problems)

1. LC 496 — Next Greater Element I
2. LC 503 — Next Greater Element II
3. LC 739 — Daily Temperatures
4. LC 84 — Largest Rectangle in Histogram
5. LC 901 — Online Stock Span

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Slow
Previous Greater Search	$O(n^2)$	$O(1)$	Slow
Monotonic Stack	$O(n)$	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)



The stack maintains decreasing stock prices. When a higher price arrives, all smaller or equal prices become part of its span and are removed. Since each price is pushed and popped at most once, the solution runs in amortized $O(1)$ time.

18. Variations / Follow-Up Questions

Follow-Up 1

What if we need:

Next Greater Price

Use Monotonic Stack.

Follow-Up 2

What if stock prices arrive as a stream forever?

This exact design already supports streaming data.

Follow-Up 3

What if we need average span over the last k days?

Combine queue + stack techniques.